基于龙芯教育派的 PCB 焊点智能缺陷检测系统设计报告

摘要

本项目面向 PCB 焊接质量检测场景，设计了一套运行在国产龙芯教育派开发板上的低成本自动光学检测系统。系统以 XYZ 三轴丝杆滑台为运动平台，以微距工业相机为视觉输入，以 YOLOv8 目标检测模型为缺陷识别核心，并通过 FastAPI、WebSocket 与原生 HTML/JavaScript 前端构建局域网 Web 工控台。整体方案希望在国产嵌入式平台、有限边缘算力和低成本机械结构条件下，完成 PCB 板件的分块扫描、图像采集、缺陷推理、结果落盘和实时可视化。

与传统 AOI 设备依赖高刚性光机结构和较高成本视觉平台不同，本项目采用 Step-and-Scan 的分块扫描方式：系统先根据 PCB 长宽和相机视场规划二维网格，再驱动 X-Y 滑台按蛇形路径移动；每到达一个网格点后执行短时间机械消抖，随后调用检测脚本完成 YOLOv8 单图推理，并将 `result.json` 中的缺陷账本通过 WebSocket 推送到浏览器端。项目当前已经形成算法训练与推理、Web 中控台、网格规划、Mock 联调、相机触发接口和电机控制接口等完整工程雏形，滑台 GPIO 驱动、相机真实抓拍和整机闭环仍处于进一步联调阶段。

关键词：龙芯教育派；LoongArch64；PCB 缺陷检测；YOLOv8；三轴滑台；TB6600；FastAPI；WebSocket

1. 项目背景

PCB 焊接缺陷会直接影响电子产品的可靠性。常见缺陷包括干焊点、少锡、多锡、短路、元件反向、安装不当和板面损伤等。人工目检在小规模实验中成本低，但存在主观性强、疲劳影响明显、检测结果难以记录和复现的问题；商业 AOI 设备性能较好，但整机价格、调试成本和维护门槛较高，并不适合本科生竞赛、小型实验室和低成本教学场景。

本项目的出发点不是简单做一个“图片上传后检测”的算法 Demo，而是尽量复现一套真实 AOI 系统从运动定位到视觉推理再到工控可视化的完整链路。目前定位为“从底层物理控制、边缘机器视觉推理，直达上层多端分布式工控看板的全栈通路”。结合当前 `worktrees` 中的代码实现，可以将项目理解为一个面向国产嵌入式平台的 PCB AOI 原型系统。

2. 设计目标

本系统的设计目标可以分为功能目标、工程目标和竞赛展示目标三类。

功能目标如下：

1. 支持输入 PCB 长宽与扫描原点，自动生成覆盖整块板件的检测网格。

2. 支持 X-Y 平面自动扫描和 Z 轴点动对焦。
3. 支持使用 YOLOv8 识别 7 类 PCB 焊接或装配缺陷。
4. 支持将检测结果保存为图片和结构化 JSON。
5. 支持通过局域网浏览器实时观察扫描状态、坐标、进度和缺陷列表。

工程目标如下：

1. 以龙芯教育派作为主控平台，验证国产处理器和 LoongArch64 Linux 环境下的边缘视觉应用可行性。
2. 使用 FastAPI 和原生前端替代本地 Qt 等重型界面，降低嵌入式端运行负担。
3. 将算法、驱动和集成控制通过固定文件和函数接口解耦，便于团队并行开发。
4. 保留 Mock 联调模式，使没有滑台、相机和权重文件时也能验证上层状态机。
5. 真实记录当前项目边界，避免把尚未闭环的驱动和相机部分写成已经完全成熟的量产方案。

竞赛展示目标如下：

1. 展示国产开发板承担工业视觉边缘控制任务的可能性。
2. 展示 “机械扫描 + 神经网络检测 + Web 工控台” 的端到端系统。
3. 展示本科生团队在算法、硬件和软件集成上的分工与接口设计能力。

2.1 前期基础工作

在进入上层算法和 Web 控制台开发之前，项目首先完成了两类底层基础工作：一类是操作系统与外设驱动层面的打通，另一类是 Python 科学计算环境在 LoongArch64 平台上的搭建。这些工作虽然不直接体现在最终 UI 页面中，但决定了后续相机采集、图像处理和模型推理能否在龙芯教育派上真正运行。

2.1.1 操作系统底层：定制内核与 V4L2 相机通路

项目启动早期，龙芯教育派的部分外设支持并不完整，尤其是板载无线网络和工业相机链路都依赖底层内核、驱动与设备树配置。团队没有停留在应用层规避问题，而是直接进入 Linux 内核构建流程，通过配置裁切、本地源码编译和内核镜像替换，为开发板准备了更适合本项目外设组合的定制内核。

这一阶段的关键收益有两点。第一，板载无线网卡被点亮后，FastAPI Web 控制台才能稳定提供局域网访问能力，手机和其他电脑可以通过同一网络访问检测页面。第二，V4L2（Video for Linux 2）相机接口被打通后，系统具备了以 Linux 标准视频设备接口访问 OV9281 工业微距相机的基础。V4L2 是 Linux 内核媒体子系统中面向视频设备的标准 API，后续 OpenCV 的 `VideoCapture` 和相机采集脚本都建立在这一底层能力之上。

从系统设计角度看，这一步相当于完成了“视觉输入通道”的物理换血：只有当内核能够识别并驱动相机设备，上层的 `camera_capture.py`、`camera_detect.py` 和 Web 扫描流程中的 `latest.jpg` 才有可靠图像来源。系统直接面向单通道黑白灰度原生矩阵，本质上也是为了减少 RGB 转换和多余数据搬运，让嵌入式平台把有限算力更多留给检测模型。

2.1.2 算力引擎中枢：Python 3.12 与跨架构依赖构建

在定制内核稳定之后，项目进一步搭建了独立的 Python 3.12 虚拟环境，用来隔离系统 Python 与项目依赖。官方 Python 文档中，`venv` 本身就是用于创建轻量虚拟环境的标准模块；对于本项目而言，虚拟环境的意义不只是方便安装包，更重要的是把 OpenCV、NumPy、Ultralytics、FastAPI 等依赖固定在一套可复现的运行上下文中，减少系统全局库变动带来的不确定性。

龙芯 LoongArch64 平台的难点在于：许多开源科学计算库在主流 x86 平台上可以直接安装二进制 wheel，但在国产架构上经常需要从源码构建。团队围绕 Python 3.12、NumPy 1.26.4 和 OpenCV 4.9.0 完成了跨架构适配，包括处理 Meson 构建断言、调用系统 GCC 编译器、补齐底层依赖，并让 OpenCV 的 C++ 动态链接库与 Python 运行环境正确咬合。NumPy 官方文档也明确指出，源码构建需要系统级编译器、BLAS/LAPACK 等依赖配合；这与项目实际遇到的问题一致。

这一阶段的价值在于打通了“图像矩阵 -> OpenCV 处理 -> YOLOv8 推理”的算力链路。NumPy 提供基础矩阵运算能力，OpenCV 提供图像读取、相机访问、绘框和图像保存能力，Ultralytics YOLOv8 则承担缺陷检测。LSX 128 位向量指令集的本地适配意味着团队不仅在应用层安装依赖，还在尝试让底层数值计算更贴近龙芯本地硬件能力。考虑到当前仓库中已经存在 YOLOv8 训练、检测、相机采集和 Web 调度代码，这些前期环境工作可以看作整套系统能够继续向真实边缘部署推进的基础。

需要说明的是，本报告对“亚秒级推理”等性能表述保持审慎态度：项目目标是尽可能实现边缘端快速推理，但最终速度仍应以龙芯板上真实运行的计时结果为准。后续可在 `detect.py` 或 `camera_trigger.py` 中加入单图推理耗时统计，将前期环境优化的效果量化为可展示的数据。

3. 需求分析

3.1 被检测对象

系统面向单块 PCB 的局部或整板检测。当前代码中相机视场被抽象为 `30 mm x 30 mm`，滑台 X-Y 最大有效行程被设置为 `135 mm x 135 mm`。因此，对于常见小尺寸实验 PCB，可以通过多网格覆盖完成检测；对于尺寸超过行程或视场规划边界的板件，系统会在网格预览和启动扫描阶段给出错误。

3.2 缺陷识别需求

算法配置文件 `worktrees/algorithm-ai/algorithm/configs/pcb_defects.yaml` 定义了 7 类缺陷：

ID	英文类别	中文含义
0	`dry_solder_joint`	干焊点
1	`improper_installation`	安装不当
2	`pcb_damage`	PCB 损伤
3	`short_circuit`	短路
4	`insufficient_solder`	少锡
5	`reverse_component`	反向
6	`excess_solder`	多锡

这些类别覆盖了焊点形态异常、元件装配异常和板面损伤三类问题。输出结果不仅要有类别，还要有检测框坐标和置信度，便于在前端形成缺陷账本。

3.3 控制与交互需求

根据 `worktrees/ui-integration/static/index.html` 和 `app.js`，前端控制台需要支持：

1. WebSocket 在线状态显示。
2. PCB 长度、宽度、原点坐标和抓拍路径输入。
3. Mock 联调模式开关。
4. 行数、列数、总网格数显示。
5. Z 轴上调、下调和点动距离设置。
6. 三轴回零、自动盲扫和停止扫描。
7. 滑台驱动、抓拍图片、YOLO 权重、检测脚本和局域网地址状态显示。
8. 网格画布、XY 坐标读数、扫描进度条、工艺事件流和缺陷账本。

这说明系统不是单纯的后端服务，而是已经有面向操作者的完整中控台雏形。

4. 总体架构

系统按照“硬件层、驱动层、算法层、集成层、交互层”五个层次组织。

操作者浏览器

|

```
    | HTTP / WebSocket
    v
FastAPI Web 中控服务
    |
    |-- 网格规划: build_grid_plan / iter_snake_cells
    |-- 扫描状态机: run_scan
    |-- 系统状态: system_status / network_status
    |
    |-- 电机接口: home_all_axes / move_z_step / move_to_xy
    |       |
    |       v
    |   GPIO + TB6600 + 三轴丝杆滑台
    |
    |-- 相机与算法接口: capture_detect_and_read
            |
            v
        YOLOv8 detect.py
            |
            v
        result.jpg / result.json
```

该结构中最核心的接口契约是：

模块	对外接口	契约
算法模块	`detect.py --weights ... --image ...`	输入单张图片，输出 `result.jpg` 与 `result.json`
相机触发模块	`capture_detect_and_read(...)`	调用检测脚本，读取最新 JSON 并返回给扫描状态机
电机模块	`home_all_axes()`、`move_z_step()`、`move_to_xy()`	上层只关心回零、对焦和定位，不关心底层 GPIO 细节
集成控制	`run_scan(...)`	串联移动、消抖、推理和结果推送
前端页面	WebSocket packet	接收 `grid_plan`、`scan_phase`、`scan_result` 等事件并渲染

这种架构很适合学生团队：算法同学、硬件同学和集成同学不需要每一步都同步修改同一个文件，只要保持接口契约不变，各自模块就可以并行推进。

5. 硬件平台设计

5.1 主控平台：龙芯教育派

系统采用国产龙芯教育派 2K1000LA 开发板作为主控。龙芯教育派基于龙芯 2K1000 系列处理器，具有千兆网络、USB、HDMI、TTL UART、多路 GPIO 和 Type-C 供电等资源。软件环境运行于 LoongArch64 国产自主架构 Linux，实际使用 Ubuntu 24.04，并围绕无线网卡、V4L2 相机驱动和 Python 科学计算依赖做了适配。

对本系统而言，龙芯教育派主要承担四类任务：

1. 运行 FastAPI Web 中控服务。
2. 通过 GPIO 或后续驱动接口控制 TB6600。
3. 通过 USB/V4L2 接入微距相机。
4. 执行 OpenCV 图像处理和 YOLOv8 推理脚本。

由于平台算力和生态都比较受限，系统没有采用复杂桌面程序，而是选择浏览器端 UI 和轻量后端服务。这一选择符合嵌入式平台轻量化和稳定性的要求。

5.2 视觉采集单元

视觉硬件采用 OV9281 工业微距相机，并通过 V4L2 直接抓取单通道灰度图像。灰度相机对焊点检测是合理选择：焊锡缺陷主要表现为亮度、边缘、形态和局部连通关系异常，并不强依赖颜色。灰度输入还可以减少 RGB 转换和带宽压力。

当前算法代码中也有对应设计：`convert_images_to_grayscale.py` 可以将数据集图片转为三通道灰度图，既保留 YOLO 训练所需的三通道格式，又使训练输入更接近真实灰度相机输出。

5.3 三轴丝杆滑台

三轴滑台具有较完整的机械定位结构。该滑台为 XYZ 三轴 PLC 丝杆定位滑台，X、Y、Z 三轴行程均为 135 mm。整体尺寸约为 X 向 290 mm、Y 向 265 mm、Z 向最高安装位置 370 mm。X 轴、Y 轴一、Z 轴使用 2040 铝材，Y 轴二和连接杆使用 2020 铝材；导向结构为 8 mm 镀铬光轴，丝杆为 8 mm 不锈钢 T 型丝杆，导程 8 mm。

该滑台的一个关键特点是双 Y 轴结构。两个 Y 轴均有电机和丝杆，两根连接杆连接两个 Y 轴以提高稳定性。X 轴额外加入导向光轴，用于提升 X 轴强度，减小 Z 轴和相机重量造成的下弯。滑台本身不适合雕刻等高负载加工，但用于承载微距相机和执行 PCB 扫描是合理的。

5.4 电机驱动与供电

项目已准备三个 TB6600 步进电机驱动器和 24 V 电源。TB6600 通常通过 DIR 方向信号和 PUL 脉冲信号控制步进电机，适合与开发板 GPIO 组成低成本运动控制方案。当前 `motor_controller.py` 中的设计也体现了这一点：代码中配置了 `X_DIR`、`X_PULSE`、`Y_DIR`、`Y_PULSE`、`Z_DIR`、`Z_PULSE` 等 GPIO 名称，并尝试通过 `gpiod` 申请 GPIO line 后输出脉冲。

需要说明的是,当前电机控制代码仍是接入中的半成品:`ui-integration/main.py` 的 `system_status()` 明确将电机状态标记为 `driver_mode: mock_stub`,并提示“未接入真实滑台驱动”;`motor_controller.py` 中也存在 Python 语法层面的待修正写法，例如 `else if` 和 `||`。因此在报告中应将该部分表述为“控制接口已定义，GPIO/TB6600 驱动正在接入与标定”，而不是表述为已经完成可靠闭环控制。

6. 软件工程结构

`worktrees` 目录体现了项目的分支化开发结构：

worktree	主要内容	说明
`algorithm-ai`	`algorithm/configs`、`algorithm/core`、`algorithm/tools`	视觉算法、训练评估和数据处理
`hardware-sm`	`motor_controller.py`	步进电机与 GPIO 控制尝试
`ui-integration`	`main.py`、`camera_trigger.py`、`static`	Web 控制台和系统集成主线
`main`	README / LICENSE	主分支工作区，当前内容较少

这种组织方式对应团队分工中的三个方向：算法层、驱动层和集成总控层。相比把所有功能都堆在一个脚本里，worktree 分工更接近真实团队协作方式。

7. Web 中控后端设计

7.1 服务启动与局域网访问

`worktrees/ui-integration/LAN_ACCESS.md` 给出了启动方式：

```
python -m uvicorn main:app --host 0.0.0.0 --port 8000
```

后端监听 `0.0.0.0:8000`，前端页面由 FastAPI 静态托管。手机和其他电脑只要与主控处于同一局域网，就可以访问页面右侧显示的局域网地址。这种方式适合比赛现场演示，也适合多人同时观察系统运行状态。

7.2 系统状态接口

`main.py` 中的 `system_status()` 汇总了电机、相机、算法、扫描任务和网络状态。它检查的关键文件包括：

状态项	对应路径或来源
抓拍图片	`algorithm/outputs/camera/latest.jpg`
YOLO 权重	`runs/detect/pcb_defects/weights/best.pt`
检测脚本	优先 `algorithm/scripts/detect.py`，否则 `algorithm/core/detect.py`
检测结果	`algorithm/outputs/detections/result.json`
局域网地址	`socket.getaddrinfo`、UDP probe、`ipconfig` 或 `hostname -I`

前端在真实模式启动前会调用 `/api/system/status` 做预检查。如果抓拍图片、权重或检测脚本缺失，就不会启动真实扫描，而是提示 “真实模式未启动”。这是一种比较稳妥的工程保护。

7.3 网格规划

网格规划由 `build_grid_plan()` 完成，核心常量如下：

常量	值	含义
`FOV_WIDTH_MM`	30.0	单次相机视场宽度
`FOV_HEIGHT_MM`	30.0	单次相机视场高度
`MAX_X_MM`	135.0	X 轴最大规划行程
`MAX_Y_MM`	135.0	Y 轴最大规划行程

行列数计算方式为：

```
cols = ceil(pcb_length_mm / 30)
rows = ceil(pcb_width_mm / 30)
```

函数还会计算 `covered_mm`、`overflow_right_mm`、`overflow_bottom_mm`、`last_col_visible_ratio` 和 `last_row_visible_ratio`。这些信息不是为了控制电机，而是为了让前端画布能标注边缘视场的超出部分。这个细节说明系统已经考虑到真实 PCB 尺寸不一定刚好是视场整数倍。

7.4 蛇形扫描路径

`iter_snake_cells()` 负责生成扫描路径。偶数行从左到右，奇数行从右到左，形成蛇形轨迹。这样可以避免每扫描完一行都回到左侧起点，减少空行程和机械往返冲击。

每个 cell 包含：

```
{
  "row": 0,
  "col": 0,
  "x_mm": 0.0,
  "y_mm": 0.0
}
```

坐标由扫描原点加上列号和行号对应的视场间距得到，即：

```
x = origin_x + col * 30
y = origin_y + row * 30
```

7.5 扫描状态机

扫描主流程位于 `run_scan()`。单个网格点的状态转换为：

```
move_xy -> debounce -> capture_inference -> result
```

具体过程为：

1. 发送 `scan_phase: move_xy`，调用 `motor_controller.move_to_xy(x_mm, y_mm)`。
2. 发送 `scan_phase: debounce`，等待 `DEBOUNCE_SECONDS = 0.05` 秒。
3. 发送 `scan_phase: capture_inference`。
4. 如果是 Mock 模式，调用 `mock_capture_detect_and_read()`。
5. 如果是真实模式，调用 `camera_trigger.capture_detect_and_read()`。
6. 检测成功后发送 `scan_result`，其中包括行列号、坐标、进度、JSON 路径和缺陷列表。
7. 所有网格完成后发送 `scan_complete`。

状态机设置了 `scan_lock`，避免重复启动多个扫描任务；设置了 `active_stop_event`，用于响应停止按钮；WebSocket 断开时也会触发停止事件。这些机制让系统更接近真实工控软件，而不只是顺序执行脚本。

7.6 Mock 联调模式

Mock 模式由 `mock_capture_detect_and_read()` 实现。它模拟约 0.35 秒推理延迟，并在 `(row + col) % 5 == 3` 时生成一个虚拟焊点异常。这一设计使前端画布、进度条、缺陷账本和停止逻辑可以在没有相机、滑台和模型权重时先运行起来。

对于本科生团队项目，这一点非常实用：算法、硬件和前端的开发节奏通常不同，Mock 模式能显著降低集成阶段的等待成本。

8. 相机与检测触发链路

`camera_trigger.py` 是 Web 集成层和算法层之间的桥。它定义了三个关键默认路径：

名称	路径
DEFAULT_IMAGE	algorithm/outputs/camera/latest.jpg
DEFAULT_WEIGHTS	runs/detect/pcb_defects/weights/best.pt
RESULT_JSON	algorithm/outputs/detections/result.json

真实模式下，`capture_detect_and_read()` 会先删除旧的 `result.json`，然后用当前 Python 解释器异步启动检测脚本：

```
python detect.py --weights best.pt --image latest.jpg
```

如果图片不存在,会返回 `capture_image_missing`;如果检测进程失败,会返回 `detect_process_failed`，并携带 stderr 尾部信息；如果进程成功但没有重新生成 JSON，会返回 `result_json_missing`。这些错误信息最终会通过 WebSocket 推送到前端。

该链路的优点是接口清晰，缺点是每个网格点都要启动一次 Python 子进程和加载一次模型，实时性可能较弱。后续如果要提升速度，可以把 YOLO 模型改为常驻进程或服务，但在当前阶段，子进程方式更容易调试，也更符合 “文件系统级接口契约” 的设计。

9. 视觉算法设计

9.1 数据集配置

算法模块默认读取 `algorithm/configs/pcb_defects.yaml`。配置中数据集路径为：

```
../../datasets/pcb_defects
```

训练集、验证集和测试集分别配置为：

```
train: images/train
val: images/val
test: images/val
```

README 中说明当前没有独立 test 划分，因此 test 暂时复用 val。这在报告中需要如实说明：当前数据集划分更适合原型验证，若要做正式效果评估，应补充独立测试集。

9.2 标注转换与数据增强

`algorithm/tools` 下提供了四个辅助工具：

文件	作用	贴合场景
`labelme_to_yolo.py`	将 LabelMe 矩形标注转为 YOLO bbox 标签	早期人工标注数据迁移
`polygon_labels_to_bbox.py`	将 YOLO polygon/segmentation 标签转为 bbox 标签	使用检测模型而非分割模型时处理标注
`augment_dataset.py`	生成增强图像并复制标签	扩充少量 PCB 缺陷样本
`convert_images_to_grayscale.py`	将图片转为三通道灰度	贴近 OV9281 灰度相机输入

增强方式包括亮度/对比度扰动、随机噪声、轻微模糊、灰度化和锐化。它们主要模拟真实拍摄中的光照、焦距和噪声变化，不改变缺陷位置，因此可以直接复用原 YOLO 标签。

9.3 模型训练

`train.py` 使用 Ultralytics YOLO 接口进行训练。训练脚本先通过 `build_runtime_data_config()` 生成运行时数据配置，再加载 `yolov8n.pt`。当前训练参数为：

参数	值
初始模型	`yolov8n.pt`
epochs	100
imgsz	960
batch	8
project	`runs/detect`
run name	`pcb_defects_` 年月日 `_` 时分秒

选择 YOLOv8n 的原因比较明确：它模型规模较小，更适合边缘平台和快速迭代。输入尺寸设为 960，是为了尽量保留焊点小目标细节，但这也会增加推理开销。实际部署时可以根据龙芯板运行速度在精度和速度之间重新权衡。

9.4 模型评估与批量预测

`evaluate.py` 会自动寻找最新的 `runs/detect/pcb_defects*/weights/best.pt`，调用 `model.val()` 在指定 split 上评估，并将 `metrics.results_dict` 保存到 `metrics.json`。这为后续报告补充 mAP、Precision、Recall 等指标预留了接口。

`predict.py` 用于对图片目录或测试集批量推理，输出带框图片。它更适合离线检查模型效果；而 `detect.py` 则更适合 Web 扫描流程中的单图检测。

9.5 单图检测输出

`detect.py` 的输出是整个系统与算法模块对接的关键。它会读取图片、加载权重、运行模型，然后输出：

1. `algorithm/outputs/detections/result.jpg`
2. `algorithm/outputs/detections/result.json`

JSON 中每个检测项包含：

```
{
  "type": "short_circuit",
  "label": " 短路",
  "bbox": [120, 80, 180, 130],
  "confidence": 0.91
}
```

其中 `type` 是英文类别，`label` 是中文类别，`bbox` 是像素坐标下的左上角和右下角，`confidence` 是置信度。前端缺陷账本正是基于这些字段渲染。

9.6 相机实时检测脚本

`camera_detect.py` 支持从摄像头或视频流连续读取帧，运行 YOLO 推理，并保存 `latest.json` 与 `latest.jpg`。它还可以选择保存带标注的视频。这个脚本说明算法组已经为实时相机检测预留了路径。

不过，Web 扫描主链路当前调用的是 `camera_trigger.py` + `detect.py` 的单图检测方式，而不是直接调用 `camera_detect.py` 的持续视频模式。原因也比较合理：AOI 扫描是走停拍照式流程，单图触发更符合网格点检测逻辑。

10. 前端控制台设计

10.1 页面结构

前端由 `index.html`、`app.js` 和 `styles.css` 组成，页面标题为“单板 PCB 焊点智能缺陷检测系统”。界面结构包括：

1. 左侧导航栏：检测控制、网格画布、缺陷账本。
2. 顶部栏：系统标题、WebSocket 状态、Ping 按钮和停止按钮。
3. 动态网格画布：显示 PCB 扫描网格、当前 cell 和缺陷 cell。
4. 板件参数区：输入 PCB 长宽、原点坐标和抓拍路径。
5. Z 轴对焦区：提供上下点动和距离输入。
6. 系统状态区：显示驱动、抓拍图片、权重、脚本和局域网 URL。
7. 扫描状态区：显示进度条、当前阶段和消抖状态。
8. 底部事件流与缺陷账本：记录扫描过程和结果。

10.2 WebSocket 数据包

前端通过 `connectWs()` 连接 `/ws/control`，发送的数据包形如：

```
{
  "type": "start_scan",
  "request_id": "...",
  "payload": {
    "pcb_length_mm": 100,
    "pcb_width_mm": 70,
    "origin_x_mm": 0,
    "origin_y_mm": 0,
    "image_path": "algorithm/outputs/camera/latest.jpg",
    "capture_mode": "mock"
  }
}
```

后端返回的数据包包括 `ack`、`grid_plan`、`scan_phase`、`scan_result`、`scan_error`、`scan_complete` 和 `scan_stopped`。前端 `handlePacket()` 根据不同 packet 类型更新状态、画布、进度和缺陷列表。

10.3 网格画布渲染

前端 `drawCanvas()` 根据后端返回的 `plan` 绘制网格。它会：

1. 按行列数计算 cell 尺寸。
2. 用不同颜色区分正常 cell、缺陷 cell 和当前 cell。
3. 对最后一行或最后一列的不可见溢出区域绘制斜线阴影。
4. 在每个 cell 中显示 `row,col`。
5. 根据 `state.results` 判断哪些 cell 有缺陷。

这个设计使操作者可以直观看到扫描进度和缺陷分布，不必只看终端输出。

10.4 真实模式预检

前端启动扫描前会调用 `refreshSystemStatus()`。如果 Mock 模式没有勾选，它会检查：

1. 抓拍图片是否存在。
2. YOLO 权重是否存在。
3. 检测脚本是否存在。

只有三者都就绪才会发送真实扫描指令。这一预检逻辑与后端状态接口共同构成了较完整的防呆设计。

11. 电机控制接口与当前边界

当前系统的电机控制接口已经在 `motor_controller.py` 中定义：

```
home_all_axes()
move_z_step(direction, dist)
move_to_xy(x, y)
```

其中 `move_z_step()` 负责 Z 轴对焦点动，`move_to_xy()` 负责 X-Y 平面移动。代码思路是使用 `gpiod.Chip` 打开 GPIO chip，再申请不同 GPIO line 输出方向和脉冲。`hardware-sm` 版本中行程按 135 mm 处理，`ui-integration` 版本中有 13.5 与 135 的单位差异，这也是后续需要统一的地方。

为了使该部分达到可比赛演示的稳定程度，需要完成以下工作：

1. 修正 Python 语法问题，将 `else if` 改为 `elif`，将 `||` 改为 `or`。
2. 统一 Web 层、驱动层和机械层的单位，明确所有距离均为 mm。
3. 根据电机步距角、TB6600 细分和丝杆导程计算 `pulse_per_mm`，替代当前较粗的 `ONE_PULSE_DISTANCE`。
4. 增加原点回零逻辑，避免只依赖函数静态变量记录当前位置。
5. 增加行程边界保护和急停逻辑。
6. 若条件允许，接入限位开关或接近开关，提升安全性和重复定位可靠性。

因此，在本设计报告中，电机部分应被定位为“运动控制接口与 GPIO 脉冲驱动方案已经形成，真实滑台驱动仍在标定和工程完善阶段”。这种表述更贴近当前代码和项目实际。

12. 运行流程

系统的一次典型运行流程如下：

1. 在龙芯教育派或主控电脑上进入 `worktrees/ui-integration`。
2. 执行 `python -m uvicorn main:app --host 0.0.0.0 --port 8000` 启动服务。
3. 浏览器打开 `http://主控 IP:8000/`。
4. WebSocket 自动连接，前端刷新系统状态。
5. 操作者输入 PCB 长度、宽度和原点坐标。
6. 前端调用 `/api/grid/preview` 获取网格规划。
7. 操作者通过 Z 轴点动完成相机对焦。
8. 若未接硬件，则保持 Mock 模式；若硬件、图片和权重就绪，则切换真实模式。
9. 点击“自动盲扫”。
10. 后端锁定扫描任务，生成 scan_id，依次执行蛇形路径。
11. 每个点完成移动、消抖、检测和结果推送。
12. 前端实时更新画布、进度条、事件流和缺陷账本。
13. 扫描结束后显示 Complete；中途点击停止则显示 Stopped。

13. 测试与验证方案

13.1 软件接口测试

建议先在 Mock 模式下测试：

1. `/health` 返回 `{"status": "ok"}`。
2. `/api/system/status` 能显示电机、相机、算法和网络状态。
3. `/api/grid/preview` 对合法 PCB 尺寸返回正确行列数。
4. 超出 135 mm 行程时后端返回 400 错误。

5. WebSocket 能收到 `ack`、`grid_plan`、`scan_phase`、`scan_result`。
6. 停止按钮能触发 `scan_stopped`。

13.2 算法测试

算法测试应覆盖：

1. 数据集目录是否包含 `images/train`、`images/val`、`labels/train`、`labels/val`。
2. `polygon_labels_to_bbox.py` 转换后的标签是否符合 YOLO bbox 格式。
3. `train.py` 是否能生成 `runs/detect/pcb_defects*/weights/best.pt`。
4. `evaluate.py` 是否能生成 `metrics.json`。
5. `detect.py` 是否能对单图生成 `result.jpg` 和 `result.json`。
6. 典型缺陷类别的误检、漏检和混淆情况。

13.3 硬件驱动测试

硬件测试应分轴进行：

1. 不装 PCB 时测试 X、Y、Z 三轴方向。
2. 用小脉冲数验证电机是否正常响应 TB6600。
3. 标定每毫米脉冲数。
4. 测试边界保护，确认不会超出 135 mm 行程。
5. 测试回零重复性。
6. 长时间蛇形路径运行后检查累计误差。

13.4 整机测试

整机测试应验证：

1. 相机安装稳定，Z 轴对焦后图像清晰。
2. 每个网格点都有对应图像或检测结果。
3. 缺陷 JSON 能正确映射到前端 cell。
4. 扫描过程中 WebSocket 不断连。
5. 停止扫描后电机和子进程都能进入安全状态。
6. 整板扫描耗时满足演示要求。

14. 当前完成度评估

结合当前系统设计和 `worktrees` 工程代码，项目完成度可以概括如下：

模块	完成度	依据
项目总体设计	较完整	文档中已有国产主控、视觉、滑台、Web 工控和团队分工描述
Web 后端	较完整	`main.py` 已实现状态接口、网格规划、WebSocket 和扫描状态机
前端控制台	较完整	`index.html` 与 `app.js` 已实现参数、画布、事件流和缺陷账本
Mock 联调	已完成	后端和前端均支持 Mock 模式
YOLOv8 算法链路	较完整	具备训练、评估、预测、单图检测和相机检测脚本
数据处理工具	较完整	具备 LabelMe 转换、polygon 转 bbox、增强、灰度化
相机真实抓拍链路	部分完成	有相机采集脚本和检测触发接口，但 Web 主链路仍依赖 `latest.jpg`
电机真实驱动	接入中	接口已定义，GPIO 思路明确，但代码仍需修正和标定
龙芯环境适配	阶段性完成	已完成内核、V4L2、Python 和 OpenCV 相关适配

15. 风险与改进方向

15.1 算力风险

YOLOv8n 虽然较轻，但在龙芯教育派上进行 960 输入尺寸推理仍可能较慢。后续可以尝试：

1. 降低 `imgsz`。
2. 对模型做剪枝或量化。
3. 导出 ONNX，寻找更适合 LoongArch64 的推理后端。
4. 将模型常驻内存，避免每个网格点重复加载。

15.2 运动风险

开环步进系统可能丢步，且滑台默认无接近开关和编码器。后续应优先增加限位与回零，建立扫描前的机械坐标基准。

15.3 图像质量风险

微距检测对焦距、光照和振动敏感。系统当前设置了 50 ms 消抖，但真实消抖时间需要实验确定。建议增加环形光源或固定光源，减少环境光变化。

15.4 数据集风险

当前 test 复用 val，不足以证明模型泛化能力。后续应采集真实滑台拍摄数据，建立独立测试集，并记录每类缺陷的 Precision、Recall 和 mAP。

15.5 接口一致性风险

前端、后端和电机层需要统一方向字符串、单位和错误返回格式。尤其是 `z_jog` 前端发送 `up/down`，驱动层也应保持一致；X-Y 坐标应统一为 mm，不能混用 cm 或 0.1 mm。

16. 项目特色

本项目的特色不在于单个算法模型本身，而在于把多个工程环节真正串联起来。

第一，平台具有国产化特点。项目选择龙芯教育派和 LoongArch64 Linux，涉及内核、V4L2、Python 科学计算库和 Web 服务等多层适配，比常规 x86 环境更有挑战。

第二，系统具有较完整的 AOI 流程。它包含运动规划、机械消抖、图像检测、结构化结果和 Web 可视化，而不仅是单图推理。

第三，工程解耦清晰。算法层通过 `result.json` 与集成层对接，驱动层通过三个函数与扫描状态机对接，前端通过 WebSocket packet 与后端对接。这样的结构利于迭代，也利于比赛答辩时解释团队分工。

第四，项目诚实保留了 Mock 模式和阶段边界。Mock 模式让系统在无硬件时仍可演示上层流程；阶段边界则提醒后续工作应集中在真实驱动、相机采集和整机测试。

17. 总结

本项目围绕 “国产嵌入式平台上的低成本 PCB AOI 检测” 这一目标，完成了从硬件选型、机械扫描、视觉算法到 Web 工控台的系统设计。现有代码已经能够支撑一个比较完整的原型：算法侧具备 YOLOv8 训练、评估、预测和单图检测能力；集成侧具备 FastAPI 服务、局域网访问、网格规划、

蛇形扫描状态机、Mock 联调和 WebSocket 推送能力；前端侧具备可视化控制台和缺陷账本；硬件侧已经形成 TB6600 + GPIO 的控制思路和函数接口。

从本科生嵌入式竞赛的角度看，这份工作比较有价值的地方在于它不是只停留在算法或硬件的单点展示，而是在有限资源条件下尝试搭建一个端到端系统。后续如果继续完善真实滑台驱动、相机自动抓拍、模型部署效率和独立测试集评估，该项目可以进一步发展为可现场演示的 PCB 智能检测平台。

参考资料

1. `worktrees/algorithm-ai/algorithm/configs/pcb_defects.yaml`：缺陷类别和数据集配置。
2. `worktrees/algorithm-ai/algorithm/core/*.py`：训练、评估、预测、检测和相机采集脚本。
3. `worktrees/algorithm-ai/algorithm/tools/*.py`：标注转换、数据增强和灰度化工具。
4. `worktrees/ui-integration/main.py`：FastAPI 后端、网格规划、WebSocket 控制和扫描状态机。
5. `worktrees/ui-integration/camera_trigger.py`：检测脚本触发与 JSON 读取接口。
6. `worktrees/ui-integration/static/*`：浏览器控制台前端。
7. `worktrees/hardware-sm/motor_controller.py`：GPIO/TB6600 电机控制接口尝试。
8. `1_ 龙芯教育派用户手册-更新版.pdf`：龙芯教育派硬件资源说明。
9. Linux Kernel Documentation: Video for Linux API。
10. Python Software Foundation: venv — Creation of virtual environments。
11. NumPy Documentation: Building from source。
12. OpenCV: OpenCV 4.9.0 release entry。